

GraphQL

GraphQL is a data query and manipulation language that allows specifying what data is to be retrieved ("declarative data fetching") or modified. A GraphQL server can process a client query using data from separate sources and present the results in a unified graph.^[2] The language is not tied to any specific database or storage engine. There are several open-source runtime engines for GraphQL.

History

Facebook started GraphQL development in 2012 and released a draft specification and reference implementation as open source in 2015.^[3] In 2018, GraphQL was moved to the newly established GraphQL Foundation, hosted by the non-profit Linux Foundation.^{[4][5]}

On February 9, 2018, the GraphQL Schema Definition Language became part of the specification.^[6]

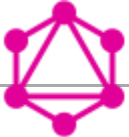
Many popular public APIs adopted GraphQL as the default way to access them. These include public APIs of Facebook, GitHub, Yelp, Shopify, Google Directions API and many others.^{[7][8]}

There is an annual GraphQL Conference^[9] featuring new developments of the protocol and organizations successfully using GraphQL. The event is hosted by the GraphQL Foundation with previous organizers incl. Prisma, Hygraph, Commercetools.

Design

GraphQL supports reading, writing (mutating), and subscribing to changes to data (realtime updates – commonly implemented using WebSockets).^[10] A GraphQL service is created by defining types with fields, then providing functions to resolve the data for each field. The types and fields make up what is known as the *schema definition*. The functions that retrieve and map the data are called *resolvers*.^[11]

After being validated against the schema, a GraphQL query is executed by the server. The server returns a result that mirrors the shape of the original query, typically as JSON.^[12]

GraphQL	
	GraphQL
Original author	<u>Meta Platforms</u>
Developer	The GraphQL Foundation
Release	September 14, 2015 (10 years, 324 days)
Stable release	September 2025 ^[1]
Written in	Implementations in <u>Java</u> , <u>JavaScript</u> , <u>Ruby</u> , <u>Scala</u> , others.
License	<u>MIT License</u>
Website	<u>graphql.org</u> (https://graphql.org)
Repository	github.com/graphql/graphql-spec (https://github.com/graphql/graphql-spec)

Type system

With GraphQL, a business domain is modeled as a graph by defining a schema; within this schema, different types of nodes and their relations are defined.^[13]

The GraphQL type system describes what data can be queried from the API. The collection of those capabilities is referred to as the service's *schema* and clients can use that schema to send queries to the API that return predictable results.^[14]

The root type of a GraphQL schema, **Query** by default, contains all of the fields that can be queried. Other types define the objects and fields that the GraphQL server can return. There are several base types, called scalars, to represent things like strings, numbers, and IDs.

Fields are defined as nullable by default, and a trailing exclamation mark can be used to make a field non-nullable (required). A field can be defined as a list by wrapping the field's type in square brackets (for example, `authors: [String]`).^[15]



Schema of swapi-graphql (<https://graphql.org/swapi-graphql/>) rendered as a graph by GraphQL Voyager (<https://github.com/graphql-kit/graphql-voyager>)

```
type Query {  
  currentUser: User  
}  
  
type User {  
  id: ID!  
  name: String!  
}
```

Queries

A GraphQL query defines the exact shape of the data needed by the client.

```
query CurrentUser {  
  currentUser {  
    name  
    age  
  }  
}
```

Once validated and executed by the GraphQL server, the data is returned in the same shape.

```
{  
  "currentUser": {  
    "name": "John Doe",  
    "age": 23  
  }  
}
```

Mutations

A GraphQL mutation allows for data to be created, updated, or deleted. Mutations generally contain *variables*, which allow data to be passed into the server from the client. The mutation also defines the shape of the data that will be returned to the client after the operation is complete.

```
mutation CreateUser($name: String!, $age: Int!) {  
  createUser(userName: $name, age: $age) {  
    name  
    age  
  }  
}
```

The variables are passed as an object with fields that match the variable names in the mutation.

```
{  
  "name": "Han Solo",  
  "age": 42  
}
```

Once the operation is complete, the GraphQL server will return data matching the shape defined by the mutation.

```
{  
  "data": {  
    "createUser": {  
      "name": "Han Solo",  
      "age": 42  
    }  
  }  
}
```

Subscriptions

GraphQL also supports live updates sent from the server to client in an operation called a subscription. Again, the client defines the shape of the data that it needs whenever an update is made.

```
subscription {  
  newPerson {  
    name  
    age  
  }  
}
```

When a mutation is made through the GraphQL server that updates the associated field, data is sent to all subscribed clients in the format setup through the subscription.

```
{  
  "newPerson": {  
    "name": "Jane",  
    "age": 23  
  }  
}
```

Versioning

While there's nothing that prevents a GraphQL service from being versioned just like any other API, GraphQL takes a strong opinion on avoiding versioning by providing the tools for the continuous evolution of a GraphQL schema.^[16]

The `@deprecated` *built-in directive* is used within the type system definition language to indicate deprecated portions of a GraphQL service's schema, such as deprecated fields on a type or deprecated enum values.^[15]

GraphQL only returns the data that's explicitly requested, so new capabilities can be added via new types or new fields on existing types without creating a breaking change. This has led to a common practice of always avoiding breaking changes and serving a versionless API.^[16]

Comparison to other query languages

GraphQL does not provide a full-fledged graph query language such as SPARQL, or even in dialects of SQL that support transitive closure. For example, a GraphQL interface that reports the parents of an individual cannot return, in a single query, the set of all their ancestors.

Testing

GraphQL APIs can be tested manually or with automated tools issuing GraphQL requests and verifying the correctness of the results. Automatic test generation is also possible.^[17] New requests may be produced through search-based techniques due to a typed schema and introspection capabilities.^[18]

Some of the software tools used for testing GraphQL implementations include Postman, Beeceptor, GraphiQL, Apollo Studio, GraphQL Hive, GraphQL Editor, and Step CI.^[19]

See also

- Query by Example
- OpenAPI Specification
- Microservices

References

1. "GraphQL September 2025 Release Notes" (<https://github.com/graphql/graphql-spec/releases/tag/September2025>). *GitHub*.
2. "Learn GraphQL Fundamentals with Fullstack Tutorial" (<https://www.howtographql.com/basics/0-introduction/>). *www.howtographql.com*. Retrieved 25 April 2023.
3. "GraphQL: A data query language" (<https://code.fb.com/core-data/graphql-a-data-query-language/>). 14 September 2015.
4. "Facebook's GraphQL gets its own open-source foundation" (<https://techcrunch.com/2018/11/06/facebook-graphql-gets-its-own-open-source-foundation/>). *TechCrunch*. Retrieved 7 November 2018.

5. "The Linux Foundation Announces Intent to Form New Foundation to Support GraphQL" (https://www.linuxfoundation.org/press/press-release/intent_to_form_graphql). *The Linux Foundation*. 6 November 2018. Retrieved 17 March 2023.
 6. "GraphQL SDL included in Github repository" (<https://github.com/facebook/graphql/pull/90/>). *GitHub*.
 7. "GraphQL Landscape" (<https://landscape.graphql.org/>). *landscape.graphql.org*. 5 July 2025.
 8. *graphql-kit/graphql-apis* (<https://github.com/graphql-kit/graphql-apis>), graphql-kit, 31 May 2025, retrieved 5 June 2025
 9. "GraphQL Conference 2025" (<https://graphql.org/conf/2025>). *GraphQL Foundation*. Retrieved 29 September 2025.
 10. "GraphQL" (<https://web.archive.org/web/20180718051011/http://facebook.github.io/graphql/June2018/#sec-Language.Operations>). *facebook.github.io*. Facebook. Archived from the original (<https://facebook.github.io/graphql/June2018/#sec-Language.Operations>) on 18 July 2018. Retrieved 4 July 2018.
 11. "Introduction to GraphQL" (<https://graphql.org/learn/>). *graphql.org*. Retrieved 25 April 2023.
 12. "Execution" (<https://graphql.org/learn/execution/>). *graphql.org*. Retrieved 25 April 2023.
 13. "Thinking in Graphs | GraphQL" (<https://graphql.org/learn/thinking-in-graphs/#its-graphs-all-the-way-down>). *graphql.org*. Retrieved 3 June 2025.
 14. "Schemas and Types | GraphQL" (<https://graphql.org/learn/schema/>). *graphql.org*. Retrieved 3 June 2025.
 15. "GraphQL" (<https://spec.graphql.org/October2021/>). *spec.graphql.org*. Retrieved 25 April 2023.
 16. "Schema Design | GraphQL" (<https://graphql.org/learn/schema-design/#versioning>). *graphql.org*. Retrieved 3 June 2025.
 17. Vargas, D. M.; Blanco, A. F.; Vidaurre, A. C.; Alcocer, J. P. S.; Torres, M. M.; Bergel, A.; Ducasse, S. (2018). "Deviation Testing: A Test Case Generation Technique for GraphQL APIs" (<https://bergel.eu/MyPapers/Mene18a-GraphQL.pdf>) (PDF). *11th International Workshop on Smalltalk Technologies (IWST)*: 1–9.
 18. Karlsson, Stefan; Causevic, Adnan; Sundmark, Daniel (May 2021). "Automatic Property-based Testing of GraphQL APIs". *2021 IEEE/ACM International Conference on Automation of Software Test (AST)*. Madrid, Spain: IEEE. pp. 1–10. arXiv:2012.07380 (<https://arxiv.org/abs/2012.07380>). doi:10.1109/AST52587.2021.00009 (<https://doi.org/10.1109%2FAST52587.2021.00009>). ISBN 978-1-6654-3567-3. S2CID 229156477 (<https://api.semanticscholar.org/CorpusID:229156477>).
 19. *GraphQL IDE Monorepo* (<https://github.com/graphql/graphiql/blob/aeedf7614e422c783f5cfb5e226c5effa46318fd/packages/graphiql/README.md>), GraphQL, 25 April 2023, retrieved 25 April 2023
- "The Apollo Studio Explorer" (<https://www.apollographql.com/docs/graphos/explorer/>). *Apollo Docs*. Retrieved 25 April 2023.
- "GraphQL Editor API Console" (<https://graphqleditor.com/docs/graphql-editor/api/>). *GraphQL Editor Docs*. Retrieved 2 September 2023.
- "Testing GraphQL APIs" (<https://docs.stepci.com/guides/testing-graphql.html>). *Step CI Documentation*. Retrieved 8 January 2023.
- "Hive Laboratory" (<https://the-guild.dev/graphql/hive/docs>). *Hive Documentation*. Retrieved 26 November 2024.

External links

- [Official website \(https://graphql.org\)](https://graphql.org)
 - [GraphQL: The Documentary \(https://www.youtube.com/watch?v=783ccP__No8\)](https://www.youtube.com/watch?v=783ccP__No8) on [YouTube](#)
-

Retrieved from "<https://en.wikipedia.org/w/index.php?title=GraphQL&oldid=1357430493>"